

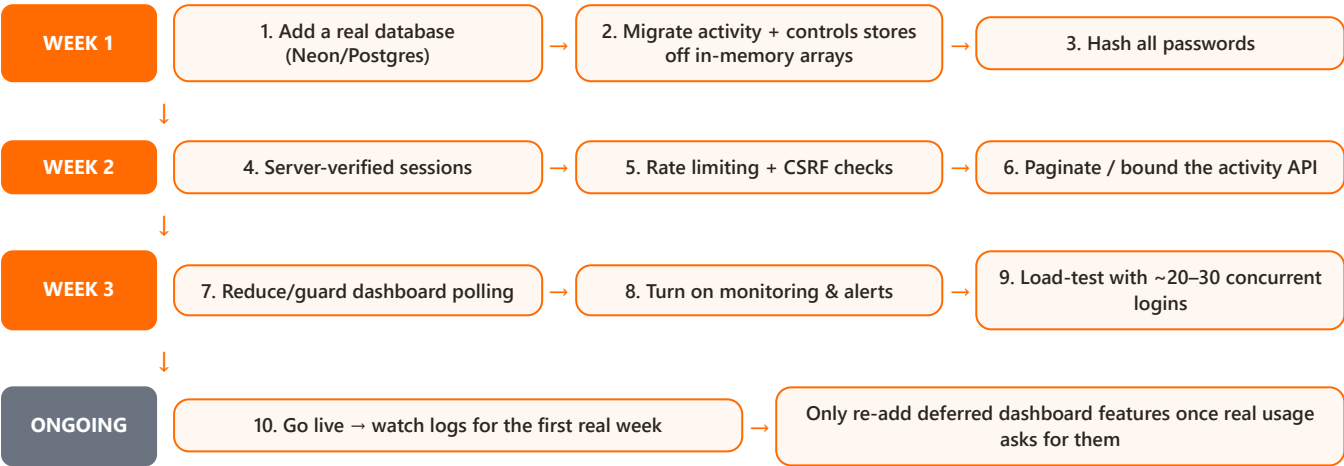
# How to Execute the Production Readiness Plan

Step-by-step execution guide for `Dashboard-Production-Readiness-Plan.pdf` — what to do, in what order, and how to know each step is done.

Prepared 2026-07-31 · Companion document to `Dashboard-Production-Readiness-Plan.pdf`

## Execution Roadmap

Each step depends on the one before it — do them in this order. Nothing in "Ongoing" starts until Week 3 is done.



Boxes = concrete step. Arrows = order of execution (left→right within a week, top→bottom across weeks).

## Week 1 — Data durability & account security P0

### Step 1: Add a real database

1. Since the app already deploys on Vercel, add a Postgres database through the Vercel Marketplace (Neon is the native option) — `vercel integration add neon`, or add it from the Vercel dashboard's Storage tab.
2. Pull the connection string into local env: `vercel env pull .env.local`. It arrives as `DATABASE_URL`.
3. Create two tables: `activity_events` (mirrors the fields in `src/lib/activity.ts`'s `ActivityEvent` type) and `staff_controls` (kicked flag + blocked project slugs per staff email).

### Step 2: Migrate the two API routes

1. In `src/app/api/activity/route.ts`: replace `globalThis.__hiranandaniActivity` with real `INSERT` (POST) and `SELECT ... WHERE` (GET) queries against `activity_events`. Keep the same filter parameters (`managerEmail`, `staffEmail`, `project`, `from`, `to`) — they map directly to `WHERE` clauses.
2. In `src/app/api/controls/route.ts`: replace the in-memory `Set` / `Map` with rows in `staff_controls`, keyed by staff email.
3. Move `seedActivity()` out of the route file into a one-time seed script (run once against the new DB), instead of regenerating on every cold start.
4. Add indexes on `staff_email`, `manager_email`, and `at` in `activity_events` — every dashboard query filters or sorts on these.

**Done when:** you can restart the dev server (or redeploy) and the activity log / kicked / blocked state from before the restart is still there.

### Step 3: Hash passwords

1. Move the 5 accounts from `src/lib/users.ts` into a `users` table, storing a password hash instead of plaintext. Node's built-in `crypto.scrypt` works with zero new dependencies.
2. Update `findUser` (currently a plain string comparison) to hash the submitted password and compare hashes.
3. Rotate the 5 demo passwords once this is live — the current ones (`admin123`, `staff123`, etc.) have been sitting in source control.

**Done when:** `git grep -i password in src/lib` shows no literal password strings.

## Week 2 — Sessions, abuse protection, and API limits P0 P1

### Step 4: Server-verified sessions P0

1. Today, `middleware.ts` and every page just check *whether a cookie is present* — nothing stops a browser from setting that cookie by hand.
2. Replace the plain cookies in `src/lib/auth.ts` with a signed, http-only session token (a JWT, or a session id that maps to a row in a new `sessions` table).
3. Verify that token server-side in `middleware.ts` on every request, instead of trusting its mere presence.

### Step 5: Rate limiting + CSRF P0

1. Add a small rate limiter (per IP + per email) in front of the login route and the activity/controls POST routes — reject after e.g. 10 requests/minute.
2. Check the `Origin` / `Referer` header on every state-changing POST and reject cross-site requests.
3. Once deployed, turn on Vercel Firewall's managed rules as a second layer.

### Step 6: Paginate / bound the activity API P1

1. In `GET /api/activity`, default to a bounded window (e.g. last 30 days) when `from` / `to` aren't supplied, instead of returning full history.
2. Add `limit` / `cursor` params for the table views in `SessionReports.tsx`.

**Done when:** a request to `/api/activity` with no filters still returns a small, fast response — not the entire table.

## Week 3 — Reduce load, add visibility, prove it holds P1 P2

### Step 7: Reduce/guard dashboard polling P1

1. The staff-profile poll (every 10s) and the general reload should pause when `document.visibilityState !== "visible"`, and/or move to a longer interval (e.g. 30s).

### Step 8: Monitoring & alerts P1

1. Turn on Vercel's built-in logs/analytics for the project.
2. Optionally wire up an error tracker (e.g. Sentry) so a crash surfaces immediately instead of via a staff complaint.

### Step 9: Load test P2

1. Write a small script that logs in and posts activity events as ~20–30 simulated concurrent staff against a staging deployment (a basic Node script hitting the API in a loop is enough — no need for a dedicated load-testing tool at this scale).
2. Watch response times, error rates, and DB connection count while it runs.

**Done when:** the simulated run completes with no errors and response times stay flat as concurrent staff increase.

## Ongoing

1. Go live, and watch logs closely for the first real week of multi-staff use.
2. Only bring back a deferred dashboard feature (roadmap diagram, extra charts, per-project blocking granularity, custom calendar, export formats, further navbar polish — see the companion plan PDF, Section 4) once a real manager or admin asks for it in daily use.

Companion to Dashboard-Production-Readiness-Plan.pdf. Re-check this roadmap once Week 1 lands — several Week 2/3 risks shrink once the database migration is done.